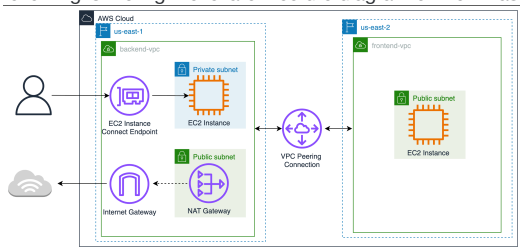


Managing Peer Connections between Amazon Virtual Private Clouds

Amazon VPCs are virtually isolated networks where you can launch resources such as EC2 instances. Amazon VPC peering connections can be used to connect two VPCs and allow them to route traffic between them using IPv4 and IPv6 addresses.

In this Cloud Lab, you will first create two VPCs: a backend VPC in the US East 1 (N. Virginia) region and a frontend VPC in the US East 2 (Ohio) region. You'll then launch EC2 instances in the private and public subnet of the backend and frontend VPC. Your backend instance will be in a private subnet, so you'll create an EC2 Instance Connect Endpoint to connect to the EC2 terminal. You'll then deploy your front- and back-end applications, but they won't be able to communicate because they're in different VPCs. Next, you'll create a VPC peering connection between your VPCs and configure their route tables to ensure traffic can flow between them.

After completing this Cloud Lab, you'll have an understanding of VPC peering connections and be able to create inter-region VPC peering connections. The following is the high-level architecture diagram of the infrastructure you'll create in this Cloud Lab:



Getting Started

A **VPC peering connection** is a networking connection between two VPCs that allows the resources inside them to communicate with each other as if they were in the same VPC. Through this service, we can access resources residing in different VPCs without exposing them to the public internet. We can create a VPC peering connection between two VPCs in the same account and region, same account but different regions, different accounts but same regions, and different accounts and different regions. In this lab, we'll create a VPC peering connection between VPCs located in the US East (Virginia) and US East (Ohio) regions.

Two things we should keep in mind while creating a VPC connection between two VPCs are that the CIDR blocks of the VPCs should not overlap, and that VPC peering does not support transitive peering. This means that if we have a VPC peering connection between VPC 1 and VPC 2, and between VPC 2 and VPC 3, this doesn't mean we can route traffic from VPC 1 to VPC 3. To do this, we'll have to create another peering connection between VPC 1 and VPC 3.

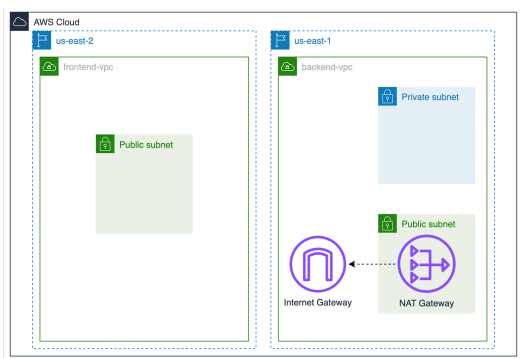
Configure VPCs

Amazon Virtual Private Cloud (VPC) allows us to create resources in a virtually isolated network. Subnets are created inside a VPC, and can be private or public. Resources created inside a public subnet can be accessed through the internet, while resources in a private subnet don't have access to the internet.

In this task, we'll create a VPC to host our back-end server in a private subnet in the **us-east-1** region. Because our back-end server requires access to the internet to install some required packages, we'll create a NAT gateway for our VPC. A NAT gateway will allow our back-end server to connect to the internet but won't allow external systems to connect to our back-end instance.

After this, we'll create another VPC to host our front-end application in a public subnet in the **us-east-2** region. We'll ensure that the CIDR blocks for our VPCs don't overlap because AWS doesn't allow us to create a VPC peering connection between VPCs and overlapping CIDR blocks.

After the completion of this task, the provisioned infrastructure should look like the following:



Architecture diagram

Let's follow the steps given below to create a VPC in the **us-east-1** region:

1. In the AWS Management Console, click on the 'VPC' link in the left-hand navigation pane.

Note: It is recommended to open the AWS Management Console in two browser windows to avoid the hassle of changing the regions again and again.

- On the AWS Management Console, click the region drop-down menu at the top right corner next to your username and select the “US East (N. Virginia) us-east-1” option.
- Search for “VPC” in the search bar and select the “VPC” service from the search results.
- From the left navigation bar, select “Your VPCs” and then click the “Create VPC” button.
- In the “VPC settings” section, do the following:
 - Select the “VPC and more” option for the “Resources to create” subsection.
 - Enter `clab-us1-backend` in the “Name tag auto-generation” field.
 - Ensure `10.0.0.0/16` is selected as the “IPv4 CIDR block.”
 - Select the “1” option in the “Number of Availability Zones (AZs)” subsection, and click the “Customize AZs” option. Select “us-east-1a” from the drop-down available under “First availability zone.”
 - Select the “1” option in the “Number of public subnets” subsection and the “Number of private subnets” subsection.
 - Select “Zonal” under the “NAT gateways (\$)” section and then select the “In 1 AZ” option for the “NAT gateways (\$)” field. This will allow the resources in our VPC to access the internet.
 - In the “VPC endpoints” subsection, select the “None” option.
 - Keep the default settings for all other options and click the “Create VPC” button.

Note: AWS might take a few minutes to create our VPC and activate the NAT gateway.

Now that we’ve created a backend VPC, open the `us-east-2` region by clicking the region drop-down menu at the top-right corner next to your username and selecting the “US East (Ohio)” option, and following the steps given above to create a VPC that will host our front-end application with the following changes:

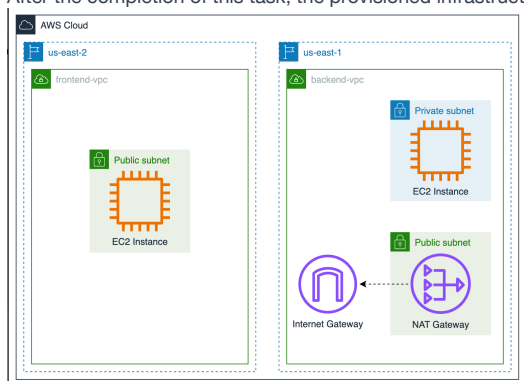
- Set its name to `clab-us2-frontend`.
- Provide `9.255.0.0/16` as its “IPv4 CIDR block.”
- Select the “1” option in the “Number of Availability Zones (AZs)” subsection, click the “Customize AZs” option, and select “us-east-2a” from the drop-down available under “First availability zone.”
- Select “1” in the “Number of public subnets” subsection, and select “0” in the “Number of private subnets” subsection.
- Select “None” in the “NAT gateways (\$)” and the “VPC endpoints” subsection.

Make sure to copy the VPC ID of the frontend VPC once it is created, as we’ll be using it later to create a VPC peering connection.

Launch the EC2 Instances

Amazon Elastic Compute Cloud (EC2) provides computing capacity inside the AWS Cloud. In this task, we’ll launch an instance in the private subnet of our backend VPC and deploy our back-end server. We’ll also launch an instance in the public subnet of our frontend VPC and deploy our front-end application.

After the completion of this task, the provisioned infrastructure should look like the figure given below:



Create security group

Before we create our EC2 instances, let’s follow the steps given below to create a security group that will control the incoming and outgoing traffic for our instance:

- Click the region drop-down menu at the top right corner next to your username and select the “US East (N. Virginia) us-east-1” option.
- Search for “EC2” in the search bar and select the “EC2” service.
- On the EC2 dashboard, select “Security groups” from the left navigation bar.
- Click the “Create security group” button.
- In the “Basic details” section, do the following:
 - Enter `clab-us1-backend-sgr` in the “Security group name” field.
 - Enter `security group for back-end vpc` in the “Description” field.

- o In the “VPC” field, select the back-end VPC we created earlier.
- Now, let’s add the following inbound rule to our security group:
 - o Click “Add rule” button under “Inbound rules” to add a rule.
 - o Select “SSH” for the “Type” field, “TCP” for the “Protocol” field, “22” for the “Port range” field, and “Anywhere-IPv4” for the “Source” field.
 - o Select “Custom TCP” for the “Type” field, “TCP” for the “Protocol” field, “3000” for the “Port range” field, and “Anywhere-IPv4” for the “Source” field.
- Ensure a rule that “All traffic” is selected under the “Type” field in the “Outbound rules” section, and then click the “Create security group” button.

Create EC2 instances

Let’s follow the steps given below to create an EC2 instance:

- On the EC2 dashboard, select “Instances” from the left navigation bar and click the “Launch instances” button.
- In the “Name and tags” section, enter `clab-backend-instance` in the “Name” field.
- In the “Application and OS Images (Amazon Machine Image)” section, select “Amazon Linux 2023 AMI” as “Amazon Machine Image (AMI)” and ensure that “64-bit (x86)” is selected as the architecture.
- Select “t3.micro” in the “Instance type” section.
- In the “Key pair (login)” section, select “Proceed without a key pair (Not recommended)” in the “Key pair name - *required*” field.
- In the “Network settings” section, click the “Edit” button and do the following:
 - o Select `clab-us1-backend-vpc` for the “VPC - *required*” field.
 - o Select “Private subnet” from the drop-down menu under the “Subnet” field.
 - o Ensure “Disable” is selected from the drop-down list under the “Auto-assign public IP” field.
 - o In the “Firewall (security groups)” subsection, click the “Select existing security group” option and select `clab-us1-backend-sgr` from the drop-down menu given under the “Common security groups” field.
- In the “Configure storage” section, ensure “8” is selected as the storage (“GiB”) and “gp3” is selected in the “Root volume” field.
- Review the settings for your instance and click the “Launch instance” button.

Once this instance is launched, change the region to `us-east-2`, and follow the steps given above to create a security group with the following configurations:

- Set its name to `clab-us2-frontend-sgr`.
- Enter `security group for front-end vpc` in the “Description” field.
- Select the front-end VPC in the “VPC” field.
- Add the following inbound rules to our security group:
 - o Select “SSH” for the “Type” field, “TCP” for the “Protocol” field, “22” for the “Port range” field, and “Anywhere-IPv4” for the “Source” field.
 - o Select “Custom TCP” for the “Type” field, “TCP” for the “Protocol” field, “3000” for the “Port range” field, and “Anywhere-IPv4” for the “Source” field.

Once you’ve created a security group, create an EC2 instance in this region with the following changes:

Note: Make sure you’re creating this instance and security group in the US East (Ohio) region.

- Set the name of this instance to `clab-frontend-instance`.
- Provide `clab-us2-frontend-vpc` as its VPC and ensure the public subnet is selected as the “Subnet.”
- Provide `clab-us2-frontend-sgr` as its security group.
- Ensure “Enable” is selected from the drop-down list under the “Auto-assign public IP” field.

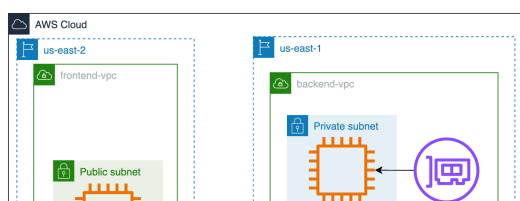
Now that we have launched EC2 instances in our public and private subnets, let’s deploy our application on these instances in the upcoming tasks.

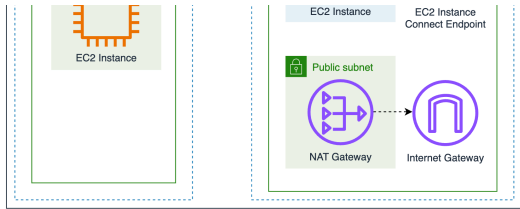
Deploy the Back-End Application

In the previous task, one of the instances we created was in the private subnet of our backend VPC in the `us-east-1` region. This instance doesn’t have a public IPv4 address, so we’ll need to create an EC2 Instance Connect Endpoint that’ll allow us to connect to our instance.

An EC2 Instance Connect Endpoint can be provisioned using the AWS service called EC2 Instance Connect Endpoint Service. This endpoint service allows users to connect to instances available in private subnets over the internet by creating a private tunnel using the instance connect endpoint.

In this task, we’ll create an EC2 Instance Connect Endpoint in our backend VPC and deploy our back-end application in the instance available in the private subnet. After the completion of this task, the provisioned infrastructure should look like the illustration given below:





Follow the steps given below to create an EC2 Instance Connect Endpoint:

Note: Make sure you're in the US East (N. Virginia) region while following the steps given below.

- Navigate to the VPC dashboard, select "Endpoints" from the left navigation menu, and click the "Create endpoint" button.
- In the "Endpoint settings" section, enter `clab-backend-connect` in the "Name tag - optional" tag, and select "EC2 Instance Connect Endpoint" for the "Type" field.
- In the "Network settings" section, select `clab-us1-backend-vpc` from the drop-down menu under the "VPC" field.
- Select `clab-us1-backend-sgr` in the "Security groups" section.
- Select the private subnet from the drop-down menu in the "Subnet" section, and click the "Create endpoint" button.

Note: AWS might take a few minutes to change the status of our endpoint to "Available." Refresh the page after a few minutes to see its status.

Once the endpoint is created, wait for the status of our EC2 Instance Connect Endpoint to change to "Available" before moving to the next step.

Deploy back-end application

Now, let's deploy this back-end application in our instance by following the steps given below:

- Navigate to the EC2 dashboard, select "Instances" from the left navigation menu.
- Select the back-end instance we created earlier, and click the "Connect" button.
- Select "Connect using a Private IP" for the connection type.
- Select the connect endpoint we created earlier in the "EC2 Instance Connect Endpoint" section, and click the "Connect" button. This will open an EC2 terminal in a new browser tab.

Now, execute the commands given below to deploy our back-end server in the EC2 terminal:

```
sudo yum -y update &&\
sudo yum -y install git &&\

sudo yum install https://rpm.nodesource.com/pub_16.x/nodistro/repo/nodesource-release-nodistro-1.noarch.rpm -y &&\
sudo yum install nodejs -y --setopt=nodesource-nodejs.module_hotfixes=1 &&\

git clone https://github.com/Educative-Content/managing-vpc-peer-connections-cl-backend.git &&\
cd managing-vpc-peer-connections-cl-backend &&\
npm install && npm start
```

These commands might take a few minutes to execute. The `Server is running on http://localhost:3000` statement will be displayed in our terminal once our back-end server is running on the port `3000`.

Note: Make sure that you don't close the EC2 terminal window because your back-end server will stop working if you do so. Also copy the IP address of the back-end instance (available below the terminal) and keep it somewhere safe since we'll be using it later.

Congratulations! We've successfully launched a back-end application server in a private subnet by connecting to the EC2 terminal using the EC2 Connect Service Endpoint.

Deploy the Front-End Application

Now that our back-end server has been deployed on an EC2 instance located in the `us-east-1` region, let's deploy our front-end application in the `us-east-2` region. This application will fetch data from our back-end application.

We created our front-end instance in a public subnet inside our VPC, so we don't need to create an EC2 Instance Connect Endpoint to connect to its terminal. Instead, we'll use the public IP address of our EC2 instance to connect to our terminal.

In the `us-east-2` region, follow the steps below to deploy our front-end application:

Note: Change the region by clicking the "region" drop-down at the top-right corner next to your username and selecting the "US East (Ohio)" option.

- Navigate to the "EC2 > Instances" page, select the front-end instance we created earlier, and click the "Connect" button.
- On the "Connect" page, click the "Connect" button. This will open an EC2 terminal in a new browser tab.
- Copy and paste the following commands in the EC2 terminal to fetch our application available in a GitHub repository:

```
sudo yum -y update &&\
sudo yum -y install git &&\

sudo yum install https://rpm.nodesource.com/pub_16.x/nodistro/repo/nodesource-release-nodistro-1.noarch.rpm -y &&\
sudo yum install nodejs -y --setopt=nodesource-nodejs.module_hotfixes=1 &&\
```

```
git clone https://github.com/Educative-Content/managing-vpc-peer-connections-cl-frontend.git &&\
cd managing-vpc-peer-connections-cl-frontend &&\
npm install && cd src && nano setupProxy.js
```

- The following file will open up in a text editor when the commands given above are executed:

```
// src/setupProxy.js
const { createProxyMiddleware } = require('http-proxy-middleware');

module.exports = function(app) {
  app.use(
    '/api',
    createProxyMiddleware({
      target: 'http://<Provide IP address of backend instance here>:3000', // Provide IP address here
      changeOrigin: true,
    })
  );
};
```

Provide the IP address of the back-end instance we copied earlier on **line 8** and save the file by pressing “Ctrl + X” (or “Control + X”), then “Y” and finally pressing the “Enter” key.

Run the application

Now that we’ve completed all the steps required to build our application, let’s finally run the application to see what it looks like. Execute the following command in the EC2 terminal:

```
npm start
```

This command will start our application. To view our application, open a new browser tab and enter the following URL. Make sure to provide the public IP address of the instance (available below the terminal) in this URL.

<http://<Provide your public IP address here>:3000>

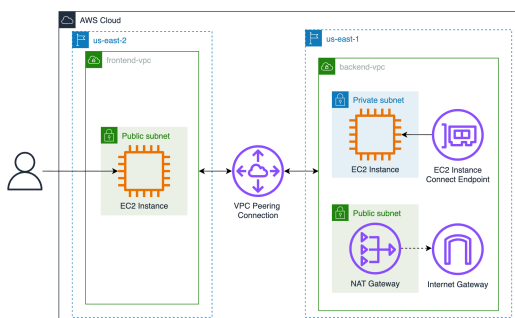
Once the application loads, you’ll see an empty message under the heading “You’ve fetched the following message from your back-end server.” This is because our front-end application server was unable to fetch data from our back-end application, as it is located in a private subnet of a VPC in the **us-east-1** region.

In the upcoming tasks, we’ll create a VPC peering connection between our two VPCs to enable them to share data.

Create a Peering Connection

A VPC peering connection is a connection between two VPCs that allows them to communicate using private IPv4 or IPv6 addresses. Once a VPC connection is established, the resources in these VPCs can communicate with each other as if they are in the same VPC.

In this task, we’ll create a VPC peering connection between the backend and frontend VPCs we created in the **us-east-1** and **us-east-2** regions, respectively. After the completion of this task, the provisioned infrastructure should look like the one given below:



In the **us-east-1** region, follow the steps given below to create a VPC peering connection request:

Note: Make sure you’re in the **us-east-1** region, before following the steps given below.

- Navigate to the VPC dashboard, select “Peering connections” from the left navigation bar, and then click the “Create peering connection” button.
- On the “Create peering connection” page, provide **clab-vpc-peer** as the name of this peering connection.
- Select the back-end VPC we created earlier from the drop-down menu available under the “VPC ID (Requester)” option.
- Ensure “My Account” is selected as the “Account” and select “Another Region” as the “Region.”
- Select “US East (Ohio) (us-east-2)” from the region drop-down menu.
- Provide the ID of the frontend VPC created in the **us-east-2** region that we copied earlier in the “VPC ID (Acceptor)” subsection.

Click the “Show Hint” button below in case you don’t have the VPC ID.

Show Hint

- Click the “Create peering connection” button to create a VPC peering connection.

Once we have created the VPC peering connection request, let’s go to the **us-east-2** region and follow the steps given below to accept this peering connection request:

Note: Make sure you’re in the **us-east-2** region, before following the steps given below.

- Navigate to the VPC dashboard and select “Peering connections” from the left navigation bar.
- Click the available VPC peering connection and then click the “Accept request” option given in the “Actions” drop-down menu.
- Click “Accept request” in the confirmation pop-up.

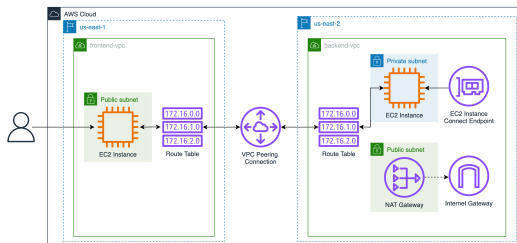
Congratulations! You’ve established a VPC peering connection between two VPCs existing in different AWS regions.

Configure Route Tables

In the previous task, we created a VPC peering connection between our VPCs located in the US East Virginia and Ohio regions. However, we’ll need to configure the route tables for these VPCs to allow them to communicate with each other.

A **route table** contains a set of routes, which are rules that define the traffic that can access our VPC. When we create a VPC, a default route table is created that is associated with all the subnets associated with our VPC. In this task, we’ll add a route to the route table of both of our VPCs which will allow them to send and receive requests using the peering connection.

We won’t be provisioning any new resources in this task, but we will be updating the configurations of the existing ones.



Let’s follow the steps given to enable communication between our VPCs by configuring their route tables:

- In the **us-east-1** region, navigate to “VPC > Route tables” and open the route table associated with our private subnet. Its name would be similar to **clab-us1-backend-rtb-private1-us-east-1a**.
- In the “Routes” tab, click the “Edit routes” button and then click the “Add route” button given on the “Edit routes” page.
- Provide **9.255.0.0/16** as the “Destination” and select “Peering Connection” from the drop-down menu given under the “Target” section.
- Select the peering connection we created earlier from the drop-down menu available in “Target” and then click the “Save changes” button.

Once this route table has been updated, let’s follow the steps given below to update the route table associated with the VPC in the **us-east-2** region.

Note: Make sure you’re in the **us-east-2** region.

- In the **us-east-2** region, navigate to “VPC > Route tables” and open the route table associated with our public subnet. Its name would be similar to **clab-us2-frontend-rtb-public**.
- In the “Routes” tab, click the “Edit routes” button and then click the “Add route” button given on the “Edit routes” page.
- Provide **10.0.0.0/16** as the “Destination” and select “Peering Connection” from the drop-down menu given under the “Target” section.
- Select the peering connection we created earlier from the drop-down available in “Target” and then click the “Save changes” button.

Once we’ve configured these routing tables, we’ll be able to transfer data between our VPCs.

Putting It All Together

Now that we’ve created a VPC peering connection between our VPCs and configured their route tables, let’s see how the data will be transferred from our back-end application to our front-end application residing in different regions.

- Our front-end application will send a request to fetch data from our back-end application.
- The route table of our backend VPC has been configured to receive requests from our frontend VPC, so our fetch request will be sent to our backend through the VPC peer connection.
- Once the backend receives this request, it will send an appropriate response.
- As the route table of the frontend VPC has been configured to receive data from our backend VPC, the response sent by the backend will be delivered to our front-end application.
- Our front-end application will display the data sent by the back-end application.

Now that we have gone over the workflow of our application, let’s check if our peering connection has been established successfully or not:

Note: Make sure that our back-end application is running in `us-east-1` region, before deploying the front-end application.

- In the `us-east-2` region, navigate to “EC2 > Instances” and connect to the instance we created earlier.
- In the EC2 terminal, run the commands given below to deploy our application:

```
cd managing-vpc-peer-connections-cl-frontend  
npm start
```

- Once our application is running in our EC2 terminal, open a new browser tab and enter the following URL. Make sure to provide the public IP address of the instance (available below the terminal) in this URL.
- <http://<Provide your public IP address here>:3000>

If our VPC peering connection was created successfully, we would be able to see the message our front-end application fetched from the back-end application deployed in the `us-east-1` region.

Clean Up

The final task is to remove all the resources we created. Open the `us-east-1` region and follow the steps given below to delete the resources we created.

Terminate the EC2 instances

- Navigate to the “EC2 > Instances” page.
- Select the instance we created earlier and click the “Instance state” option.
- Select the “Terminate instance” option from the drop-down menu.
- Click the “Terminate” button in the prompt box.

Delete the VPC endpoints

- Navigate to the “VPC > Endpoints” page and select the endpoint we created earlier.
- In the “Actions” drop-down menu, click the “Delete VPC endpoints” button.
- In the confirmation pop-up, type `delete` and click the “Delete” button.

Note: Make sure the VPC endpoint has been deleted before proceeding to the next steps.

Delete security groups

- Navigate to the “VPC > Security groups” page and select the security group we created earlier.
- Click “Actions” and select the “Delete security groups” option from the drop-down menu.
- In the confirmation pop-up, type `delete` and click the “Delete” button.

Delete NAT gateway

- Navigate to the “VPC > NAT gateway” page and select a NAT gateway.
- Click “Actions” and select the “Delete NAT gateway” option from the drop-down menu.
- In the confirmation pop-up, type `delete` and click the “Delete” button.

Delete the VPC peering connection

- Navigate to the “VPC > Peering connections” page and select the peering connection created earlier.
- Click “Actions” and select the “Delete peering connection” option from the drop-down menu.
- In the confirmation pop-up, select “Delete related route table entries” and type `delete` in the given text box.
- Click the “Delete” button.

Note: While deleting the VPC peer connection, you might see an error saying one route is not deleted. You can safely ignore it as it will be deleted when we delete our VPC.

Delete the VPC

- Navigate to the “VPC > Your VPCs” page and select the backend VPC we created earlier.
- Click “Actions” and select “Delete” from the drop-down menu.
- In the confirmation pop-up, type `delete` and click the “Delete” button.

Now, follow the steps given above in the `us-east-2` region to delete the instance, security groups, and VPC we created in that region.